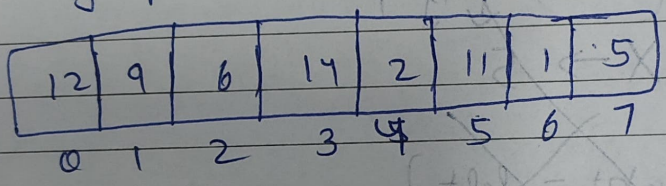
DOMS Page No.
Date / /

* Basic Concept

See we take an array.

$$\{12, 9, 6, 14, 2, 11, 1, 5\}$$

this is the array we have, now what we gotta do is break this array till only one element remains although we are breaking it into single element it is not like we are creating an single element multiple arrays, that would not be so efficient. It is a theoratical concept to write on paper, that the array is gonna divided into single element, so continuous array part. which is



See an array is a contiguous memory block, means it is an indexing format so, it will start from zero to last

Now the question comes how we gonna divide the elements into single imaginary block of array that will contain the element.

Now we might think to chop one by one the array which may seem fine, but it's maybe not implementable for this, we may require a additional space to hold on. we not gonna look into that.

The best approach we have is divide the array from

middle into it's two sub-array form till single element remains

Now question is how we gonna divide from middle,

① approah is traditional one : $\frac{\text{left + right}}{2}$ $\left(\frac{10-0}{2}\right)=5$

② approach : $\text{left} + \left(\frac{\text{right} - \text{left}}{2}\right)$

assum left = 2,000,000,000
right = 2,100,000,000

middle $\Rightarrow \frac{l+r}{2} = \left(\frac{4,100,000,000}{2}\right)$

Now, as we know C, C++, Java or C# uses a 32-bit signed interge

↓

this signed term refers to a bit used in 32 bit to track the sign of integer where 0 represents positive
1 represends negative

So max value of 32 bit signed (means positive) integer can be 2,147,483,647

as 4,100,000,000 > the max limit of 32-bit signed interger, the value will overflow means, becomes a negative number like $-\left(2,147,483,647 + \left(4,100,000,000 - 2,147,483,647\right)\right)$

$= -194,967,296$, as we are calculating the middle value to halve our array, means middle will be used

as index, and index can't be negative, this will throw an exception that is Array Index Out of Bounds Exception or even memory crash.

therefore if we know that we are working with smaller no. of elements than we can prefer it. otherwise we'd be solving technical problems go for the another one. Let's have a look onto it.

② Approach -> $left + \frac{(right - left)}{2}$

let just say

$l = 2,000,000,000$

$r = 2,100,000,000$

$\frac{r-l}{2} = \frac{100,000,000}{2} = 50,000,000$

$= 2,000,000,000 + 50,000,000$

$= 2050,000,000$, thus we are inside the boundary limit and got our middle element

Now Question is how we gonna remember it:

"start at left, then add half of the distance"

$$left + \left(\frac{right - left}{2}\right)$$

Now we done with finding the middle element. we were finding middle element so that we can halve the array into two sub-parts:
we initially had array

0	1	2	3	4	5	6	7
12	9	6	14	2	11	1	5

Now, mid will become = $0 + \left(\frac{7-0}{2}\right)$ (3.5)

mid = 3 {cause we are working with integer}

So now will halve the array into two parts.
that is:

① 0, mid

② mid+1, right

0	1	2	3
12	9	6	14

4	5	6	7
4	11	1	5

We get

0	1	2	3
12	9	6	14

4	5	6	7
4	11	1	5

Our task is to divide it until it becomes single element (imaginary)

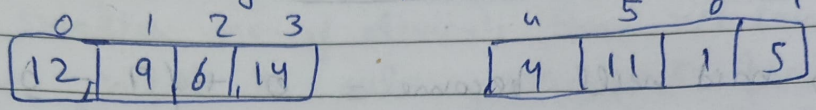
Now how this imaginary becomes reality. this is done by the term which is **Recursion**:

A recursion is function which calls itself

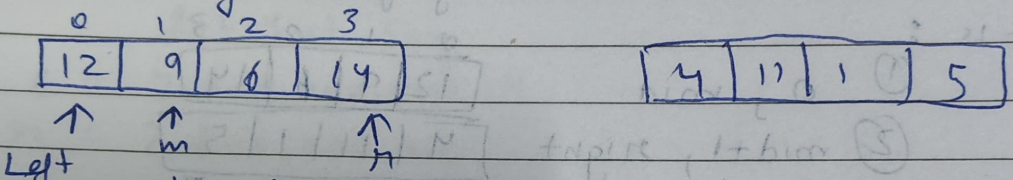
So we will create recursion until we get an single element of an array

So, the question is how we gonna create the recursive function.

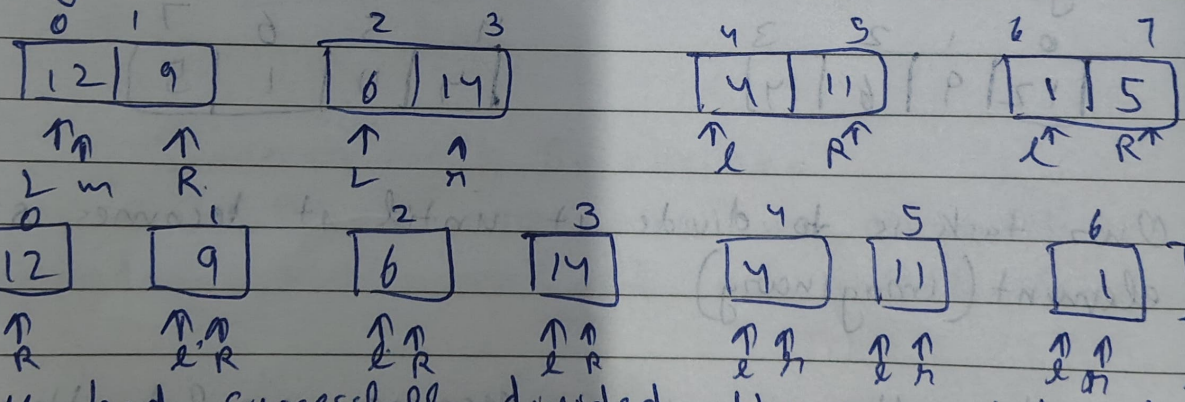
Let's have a look into it; as you know we are working with an array and dividing it into its subarrays until we get a single element of an array. Let's Imagine it, until now we done.



We First applied recursive function on whole array, Now similarly it will be done to it's sub-array, that is



again divided.



We had successfully divided the array into single subarray, Now the question is where does the recursion gonna go and stop. As our ultimate goal was to divide into single element array. So it is where the recursion gonna stop.

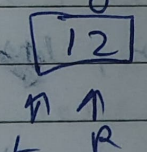
So how is recursion gonna stop?

If you people had noticed that each time l and r are being changed until they points to same element

Now it may seem at valid response to say, the base condition as if (L != R) { then do }

but it does not recline with the motive of our safe-programming - you see, to me it also seemed fine until and unless the user accidently do not put l > right. because if he/she did so it will result in infinite recursion or crash. don't worry we run this program in both the condition while giving the correct values and being carefull.

But the better version of it is to use L <= R

take the element, 

here 0 <= 0 recursion stops

or 0 != 0 (True)

↓ recursion stops

But if somehow, L = 3 and R = 1

3 != 1 , this is disaster.

We looks onto code don't worry. So up until now we had done how the recursion is gonna work and divided the elements into single element array.

Now you might wonder where does the sorting gonna occurs. Congratulations we are officially on that part. Now the path ahead is the easy one. Let's look back on the condition of the array we punctured.

0	1	2	3	4	5	6	7
12	9	6	14	4	11	1	5

For this let's write the previous code.

```
void mergesort(vector<int> arr, int lft, int right){
                                    l = lft; r = right
    if (l == R){

        mid = l + ((R - l)/2);

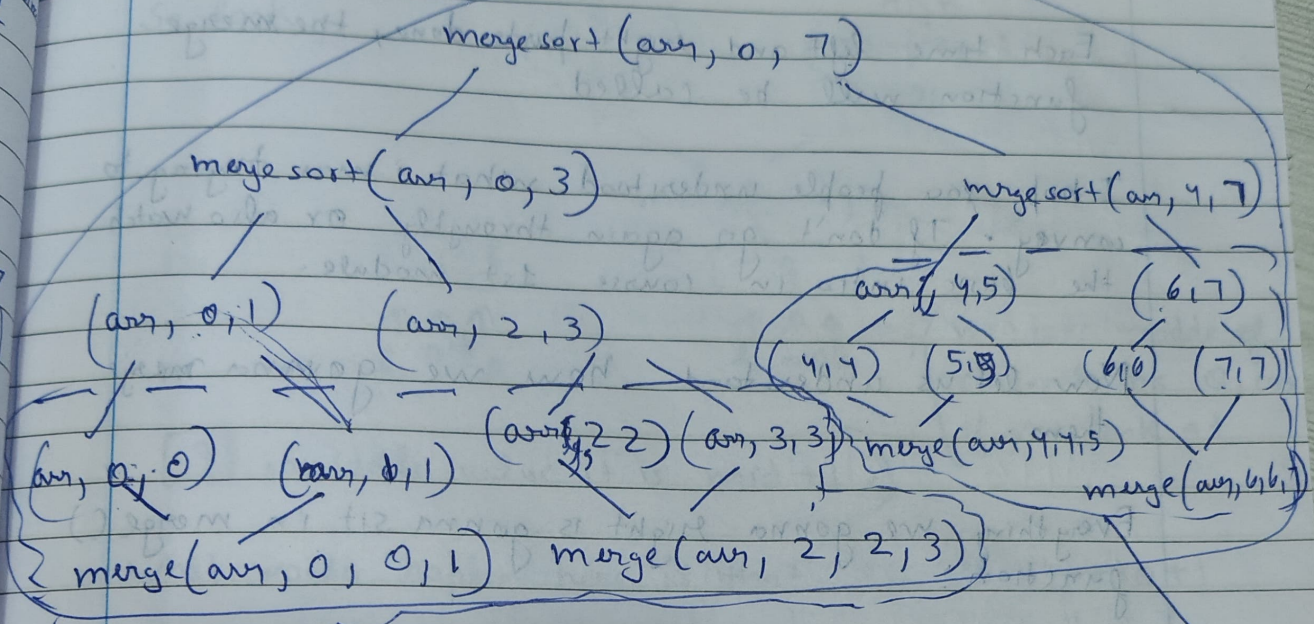
        mergesort(arr, l, mid); // lft array

        mergesort(arr, mid+1, r); // right array

        merge(arr, l, mid, r);

}
```

Recursion tree



after this execution, the function merge(arr, l, mid, r) will be called. yo use the l in here is 0, mid in here is 3, and r is 7

so merge(arr, 0, 3, 7) will be called

When this part is done again will the merge(arr, 0, 1, 3) be called.

When this called done again be called.

I hope you people understanding what iam saying. for better understanding of recursion checkout courses- "Advanced Data structures and Algorithms" by Packt.

Each time left and right parts done, the merge function will be called.

I hope you people understood what i was trying to convey. If don't go again throughly or also watch the basic intro in course 1st module.

Now let us understand how we gonna merge them.

Everything we gonna right is gonna sit in merge() function.

```
Merge(arr, left, mid, right){
```

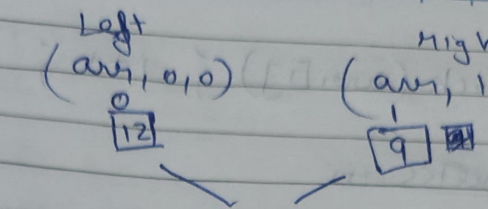
we will create an array which is temporary to store element one by one.

we will create two pointers to keep the record and comparing the element to store in the temporary array. which goes as

```
{
  vector<int> temp;
  int i = l; int j = mid+1;
}
```

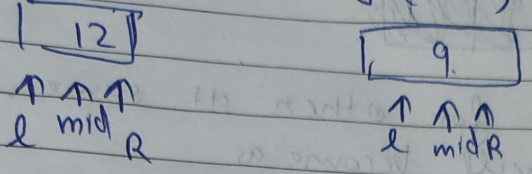
Now you might think, why the hell i initiated i as left(0) and j = mid+1 (whatever will be)

So look,



merge(arr, 0, 0, 1)

in



, (we take middle of obviously as 0, cause if you look at [9], middle value it is mid+1.)

what would happen here is that, i = 0, j = 0+1

j = 1

↓ which is the second element of array.

I hope, you understood what i tried to say.

Next part of comparison is done as :

as long as i(which is 0) is less than or equals to mid && j is less than or equal to right.

while(i <= mid && j <= r) { compare them }

For now just keep in mind we did something like this for future reference.

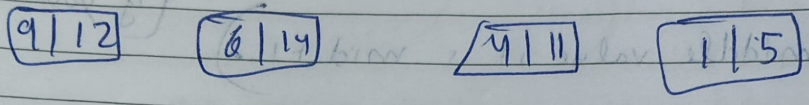
Now the process of comparison :- if the element at [i] is greater than element at [j], then push_back(arr[j]) and j++; else

push_back(arr[j]); i++;

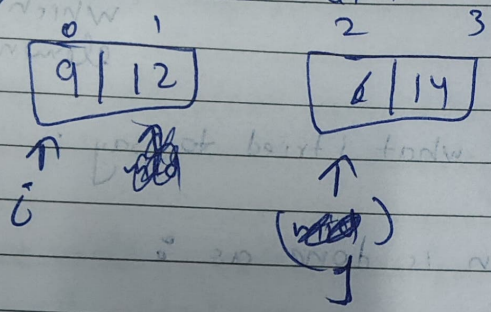
~~which results in~~

~~temp~~

and doin so for other at same level of recursion everything will become as



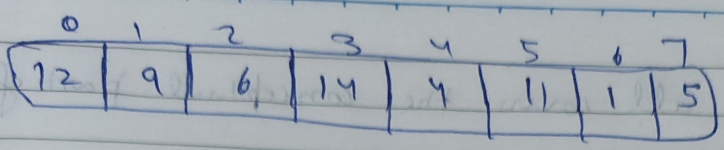
As i told earilier after each left-right execution the merge( ) function will be called.
So, lets look at it



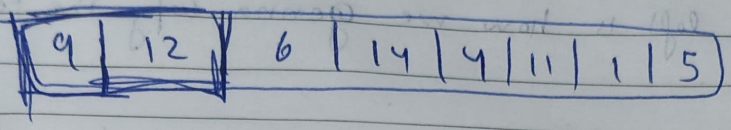
So we will compare the element at i and element

There is something to be understood that each time the merge( ) function is called, the elements on which we worked and stored in temp will be copied to the orinal array. its not like you did sort and you copy the sorted array at once to main array. it is also done recursively.

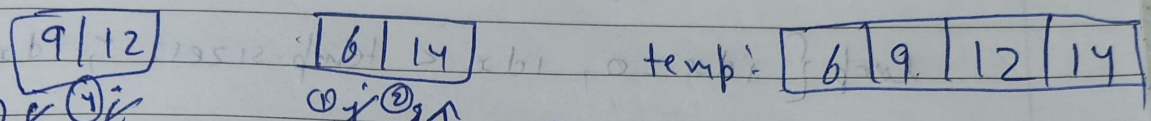
if is, say after comparison the temp is 9|12
it will be copied into original array which was



it will become

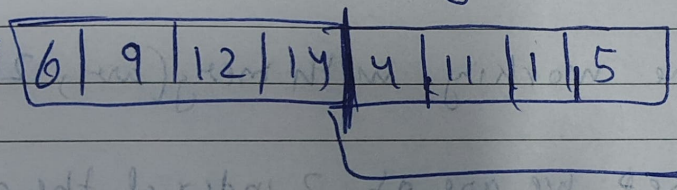


after first execution then.



put small element into temp

then main array becomes



It also done in similar manner.

Now a one more concepte, is when all the elements of one of the two sub-array has been pushed and j is still pointing to an element or number of element are left how will be gonna do it.
for that we gonna use an additional while loop.

while (j <= ~~mid~~ right) { push arr[j] and j++ } similary for left array

while (i <= mid) { push arr[i] and i++ }.

upto this, it all covers the conceptual part of merge sort.

Now the work left is how we gonna copy the temp in main array.

It is done according matching exact physical position

```
loop{ idx = 0, idx < temp.size(); idx++){
    arr[idx + start] = temp[idx]
}
```

Why we wrote idx + start, this is because, assume we are working with merg(arr, 2, 2, 3)

here you can see we are at 2 index of the array, means 0,1 had been sorted in their respective

arr[0 + 2] = temp[0]

which will store the values. That's it.

Hooray, It's completed Now. Obviously the conceptual part